

Pake Plus

基于 Pake 的网页桌面应用增强工具

实验报告

成员 A 学号 XXXXXX (广告/跟踪拦截)

成员 B 学号 XXXXXX (离线缓存代理)

成员 C 学号 XXXXXX (剪贴板管理)

成员 D 学号 XXXXXX (设置面板 + 数据迁移 + 诊断 + 视频)

2026 年 7 月 17 日

摘要

Pake Plus 是基于开源项目 Pake (Rust + Tauri v2) 的网页桌面应用增强工具。原 Pake 可将任意网页打包为约 5MB 的轻量级桌面应用 (仅为 Electron 的 1/20)，但打包产物仅具备基础窗口功能。

本项目在 Pake 基础上新增四大模块：**广告/跟踪拦截** (EasyList 规则引擎，双重过滤)、**离线缓存代理** (HTTP 层透明缓存，LRU 淘汰)、**剪贴板管理** (系统级 FFI 监听，全文搜索) 和 **设置面板** (可视化配置、数据迁移、系统诊断、版本回滚)。

技术栈为 Rust + Tauri v2 IPC + 原生 JavaScript，四个模块通过共享 JSON 配置文件实现解耦协作。Rust 后端负责系统级操作 (文件 I/O、ZIP 打包、系统信息采集、OS FFI)，JavaScript 前端通过纯 DOM 操作构建用户界面。项目总计 67 个 Rust 单元测试、297 个 JavaScript 测试，全部通过，0 warnings。

关键词：Rust; Tauri; 桌面应用; 广告拦截; 离线缓存; 剪贴板管理; WebView

目录

1	项目名称与团队介绍	5
1.1	项目简介	5
1.2	团队成员与分工	5
2	项目背景与目标	5
2.1	选题原因	5
2.2	改进目标	6
2.3	与原项目 Pake 的区别	6
3	系统设计与整体架构	6
3.1	技术栈	6
3.2	整体架构图	7
3.3	模块关系	7
3.4	文件结构	7
3.5	技术亮点	8
4	功能模块实现	8
4.1	广告拦截模块（成员 A）	8
4.1.1	设计思路	8
4.1.2	核心流程	10
4.1.3	关键代码	11
4.1.4	效果截图	14
4.1.5	遇到的问题与解决方案	14
4.2	离线缓存代理模块（成员 B）	15
4.2.1	设计思路	15
4.2.2	核心流程	15
4.2.3	关键代码	15
4.2.4	效果截图	16
4.2.5	遇到的问题与解决方案	16
4.3	剪贴板管理模块（成员 C）	16
4.3.1	设计思路	16
4.3.2	核心流程	16
4.3.3	关键代码	18
4.3.4	效果截图	20
4.3.5	遇到的问题与解决方案	20
4.4	设置面板与数据管理模块（成员 D）	24

4.4.1	设计思路	24
4.4.2	核心流程	24
4.4.3	关键代码	25
4.4.4	效果截图	27
4.4.5	遇到的问题与解决方案	28
5	实验结果	29
5.1	测试数据	29
5.2	关键测试用例	29
5.3	命令行示例	29
6	开源项目参考说明	31
6.1	引用部分	31
6.2	新增部分（对比原项目）	32
7	总结与展望	32
7.1	项目总结	32
7.2	不足与改进方向	32

1 项目名称与团队介绍

1.1 项目简介

Pake Plus是基于开源项目 Pake（Rust + Tauri v2）的网页桌面应用增强工具。原 Pake 项目可将任意网页打包为轻量级原生桌面应用（约 5MB），但功能仅限基础窗口管理和系统托盘。Pake Plus 在此基础上新增四个独立功能模块，使其更接近一个成熟的桌面浏览器应用。

1.2 团队成员与分工

成员	负责模块	核心工作
成员 A	广告/跟踪拦截	EasyList 规则引擎、网络请求拦截、DOM 元素隐藏、自定义规则、拦截统计
成员 B	离线缓存代理	HTTP 代理拦截、磁盘缓存管理、LRU 淘汰算法、离线模式
成员 C	剪贴板管理	系统剪贴板监听（FFI）、历史记录面板、全文搜索、自动清理
成员 D	设置面板 + 数据	可视化配置页面、数据导出/导入（ZIP）、系统诊断、视频制作

2 项目背景与目标

2.1 选题原因

Pake 是一个优秀的轻量级桌面应用打包工具，利用系统 WebView 代替 Electron 的 Chromium 内核，打包体积仅约 5MB（约 Electron 的 1/20）。然而，原项目在功能层面存在明显不足：打包后的应用只是一个”空壳浏览器”，缺少现代桌面浏览器应具备的以下能力：

- (1) 无广告过滤能力 — 网页中的广告和跟踪脚本原样加载
- (2) 无离线缓存 — 断网后页面完全无法访问
- (3) 无剪贴板历史 — 依赖系统剪贴板，复制内容无法回溯
- (4) 无可视化配置 — 所有参数在打包时通过 CLI 一次性写入，运行时不可修改

2.2 改进目标

本项目旨在解决上述四个痛点，在不显著增加打包体积的前提下，通过 Rust 后端 + Tauri IPC 框架，为 Pake 打包的应用增加广告拦截、离线缓存、剪贴板管理和可视化设置四大功能模块。

2.3 与原项目 Pake 的区别

对比维度	原 Pake	Pake Plus
广告处理	无任何广告过滤能力	EasyList 规则引擎，网络层拦截 + DOM 元素隐藏双重过滤
离线能力	完全依赖网络	HTTP 层代理缓存，已访问页面断网仍可浏览
剪贴板能力	无历史记录	系统剪贴板监听，全文搜索 + 一键复用
功能配置	打包时 CLI 一次性配置	运行时可视化面板，修改即时生效并持久化
数据迁移	不支持	一键导出 ZIP，换设备导入恢复
系统诊断	无	编译信息 + 系统信息，一键复制报告

3 系统设计与整体架构

3.1 技术栈

- 后端：Rust（Tauri v2框架）
- 前端：TypeScript（CLI）+ JavaScript（WebView注入）
- 桌面壳：系统 WebView（Windows: WebView2，macOS: WKWebView，Linux: WebKitGTK）
- IPC：Tauri Command（Rust JavaScript 双向调用）
- 关键 Crates: zip 2.2、sysinfo 0.33、arboard 3.4、chrono 0.4、built 0.7、clipboard-rs 0.3、rusqlite 0.32、sha2 0.10、regex 1、url 2.5、serde / serde_json

图 1: Pake Plus 系统架构（待插入图片）

3.2 整体架构图

3.3 模块关系

四个模块在功能层面完全独立，互不依赖：

- A 的广告拦截不需要 B/C/D 的任何功能即可运行
- B 的离线缓存不需要 A/C/D 的任何功能即可运行
- C 的剪贴板管理不需要 A/B/D 的任何功能即可运行
- D 的设置面板通过读/写共享配置文件管理 A/B/C 模块，但不影响各模块独立运行

四个模块通过 D 定义的 `AppSettings` 数据结构进行配置共享，通过 Tauri 的 `#[command]` IPC 机制暴露接口给前端调用。

3.4 文件结构

Listing 1: Pake Plus 代码结构

```
1 src-tauri/src/
2   adblock/                # [A] 广告拦截模块
3     mod.rs                # 模块入口 + AdblockState
4     engine.rs             # URL 匹配引擎（域名/正则）
5     rules.rs              # EasyList 规则解析器
6   app/
7     clipboard/            # [C] 剪贴板管理模块
8       mod.rs              # 模块入口
9       monitor.rs          # 系统剪贴板监听（FFI）
10      store.rs             # SQLite 存储与检索
11      filter.rs            # 隐私过滤（密码/银行卡/Luhn）
12      panel.rs             # 剪贴板历史面板窗口
13      commands.rs         # IPC 命令
14      settings.rs          # 剪贴板专用设置
15      cleanup.rs           # 自动清理
16      source.rs            # 来源应用识别
17    settings/              # [D] 设置面板模块
18      mod.rs               # 模块入口 + 重导出
```

```
19         types.rs           # 数据结构 + Theme/Language 枚举
20         traits.rs          # ModuleSettings trait
21         io.rs               # 文件读写 + 版本备份
22         health.rs           # 启动健康检查
23         diagnostics.rs      # 诊断信息采集
24         commands.rs         # IPC 命令
25         tests.rs            # 单元测试
26     setup.rs                # 托盘菜单 + 全局快捷键
27     window.rs               # 窗口创建 + JS 注入
28     ...
29 inject/
30     custom.js               # [D] 设置面板侧边栏（齿轮按钮触发）
31     settings.js            # [C] 剪贴板测试面板
32     adblock.js              # [A] fetch/XHR 拦截 + DOM 隐藏
33     event.js                # 剪贴板快捷键 + 事件处理
34     ...
35 lib.rs                     # 应用入口 + 命令注册
```

3.5 技术亮点

4 功能模块实现

4.1 广告拦截模块（成员 A）

4.1.1 设计思路

广告拦截模块采用“规则引擎 + 前端拦截”的双层架构。Rust 后端负责解析 EasyList 格式的规则文本，将其分类为域名过滤规则、正则表达式规则和 CSS 隐藏选择器三类，通过 Tauri 的 `initialization_script` 机制序列化为 JSON 注入到页面中。前端 JavaScript (`adblock.js`) 在页面加载时读取 `window.pakeAdblock` 配置，随后代理浏览器原生的 `fetch` 和 `XMLHttpRequest` 对象——对匹配黑名单的网络请求直接返回 HTTP 204 空响应，阻止广告资源加载；同时将 CSS 隐藏选择器注入页面 `<style>` 标签，并通过 `MutationObserver` 监听 DOM 变化，确保后续动态插入的广告元素也被实时隐藏。

模块的构建时配置通过 `--block-ads` CLI 选项启用，通过 `--adblock-rules <path>` 加载用户自定义规则文件（每行一条，支持 `||domain^` 域名匹配和 `##.selector` CSS 隐藏两种格式）。运行时，设置面板的 Adblock 标签提供 ON/OFF 开关和折叠式规则编辑器——开关通过全局变量 `window.__pakeAdblockRunning` 实时控制 `adblock.js` 的拦截行为（无需页面刷新）；自定义规则保存时，Rust 后端的 `save_settings` 命令自动

亮点	说明	涉及模块
Rust 枚举类型安全	Theme/Language 从 String 重构为 enum，非法值编译期拒绝，#[serde(alias)] 向后兼容旧格式	D
ModuleSettings trait 多态	每个模块独立实现 validate() / summary() / stats(), IPC 统一调用	D
跨平台剪贴板监听	Windows、macOS 与 X11 使用 clipboard_rs 原生事件监听，纯 Wayland 使用 wl_clipboard_rs 低频轮询兜底，独立线程不阻塞主界面	C
双重哈希去重与回写抑制	基于 SHA-256 的 last_hash 与 expected_hash 双重判断，过滤系统重复通知，并避免从历史面板复制时再次入库	C
隐私过滤与来源识别	识别 Windows、macOS、X11 来源应用，支持忽略应用、疑似密码检测和银行卡号 Luhn 校验，敏感内容在入库前即被拦截	C
SQLite 历史检索与自动清理	SQLite WAL 持久化、唯一哈希索引与 UPSERT 去重，支持多关键词 AND 模糊搜索，并按保留天数和容量上限定时清理	C
HTTP 层透明代理缓存	基于 Tauri 资源拦截，异步磁盘 I/O，LRU 淘汰算法	B
EasyList 规则引擎	网络请求拦截 + DOM 元素隐藏双重过滤，支持自定义规则	A
JavaScript 请求代理	代理 window.fetch / XMLHttpRequest，匹配规则返回空响应，MutationObserver 动态隐藏广告元素	A
运行时规则同步	设置面板保存自定义规则后通过 IPC 同步更新广告引擎，即时生效	A
设置版本备份与自动恢复	保存前轮转 5 个历史版本，启动时健康检查自动从备份修复损坏文件	D
模块化文件结构	800 行单文件拆为 8 个模块 (types / traits / io / health / diagnostics / commands / tests)	D
数据可移植性	动态扫描数据目录打包 ZIP，导入前预览确认，先备份后覆盖	D

表 1: 技术亮点汇总

对比新旧规则并调用引擎的 `add_custom_rule / remove_custom_rule` 完成同步。拦截计数器通过托盘 tooltip 显示，设置面板的拦截统计区域通过 `adblock_get_stats` IPC 命令获取实时数据。

技术难点在于：初始化脚本的执行时机早于页面内容加载，需要确保 `AdblockState` 在窗口构建前完成初始化（否则 `try_state` 返回 `None` 导致 `adblock.js` 不被注入）；Tauri v2 的 ACL 权限系统默认拒绝所有自定义 IPC 命令，需要将所有命令统一配置在权限文件中；CSS 隐藏选择器需要兼容 `EasyList` 的多种语法变体（`##`、`###`、`#@#` 例外规则等）。

4.1.2 核心流程

- (1) **构建时规则加载。** 用户执行 CLI 命令 `node dist/cli.js <url> --block-ads --adblock-rules ./my-rules.txt` 时，CLI 将 `block_ads` 和 `adblock_rules` 写入 `.pake/pake.json`。Rust 编译期间通过 `include_str!` 读取内置规则文件 `easylist-mini.txt`（包含常用广告和跟踪域名），与用户自定义规则合并后初始化 `AdblockEngine`。
- (2) **规则引擎初始化与状态管理。** 应用启动时，`run_app()` 首先将 `pake_config.block_ads` 和 `adblock_rules` 传递给 `AdblockState::new`。该函数调用 `AdblockEngine::from_rules_text` 解析所有规则文本：`parse_rules_text` 按行遍历，通过正则匹配识别规则类型（`||domain^` → 域名规则、`/regex/` → 正则规则、`##selector` → CSS 隐藏选择器），结果存入 `EngineRules` 结构体。初始化完成后通过 `app.manage()` 将状态注册到 Tauri 状态管理器。
- (3) **规则导出与前端注入。** 窗口构造函数 `build_window` 检查 `config.block_ads` 标志，若启用则通过 `app.try_state::<AdblockState>()` 获取引擎实例，调用 `export_for_injection(None)` 将内存中的域名列表（`Vec<String>`）、正则表达式（`Vec<String>`）、CSS 选择器（`Vec<String>`）序列化为 JSON，以 `window.pakeAdblock = {enabled:true, domains:[...], regexes:[...], cosmetic_selectors:[...]}` 格式注入为初始化脚本。随后注入 `adblock.js` 文件。
- (4) **前端网络请求拦截。** `adblock.js` 执行时首先检查 `window.pakeAdblock` 是否存在且 `enabled` 为 `true`。若启用，保存原始 `fetch` 和 `XMLHttpRequest.open` 引用，然后替换为包裹函数。包裹函数调用 `shouldBlockUrl()` 对请求 URL 进行匹配——域名规则直接比较 `hostname`（支持精确匹配和子域名匹配），正则规则对整个 URL 进行 `RegExp.test()`。匹配成功则调用 `reportBlock()` 递增计数器，然后返回 HTTP 204 空响应或替换 `XMLHttpRequest` 为 `about:blank`，阻止实际网络请求。

- (5) 前端 CSS 隐藏与 DOM 监听。 `applyCosmeticRules()` 遍历 `cosmetic_selectors` 列表，删除 `##` 和 `###` 前缀，将剩余选择器拼接为 `{display:none!important;visibility:hidden!important}` 样式，注入到页面 `<style id="pake-adblock-style">` 标签中。同时创建 `MutationObserver` 监听 `documentElement` 的 `childList` 和 `subtree` 变化，每当有新元素插入时重新应用 CSS 规则，确保动态加载的广告也被隐藏。
- (6) 运行时开关控制。设置面板的 Adblock 标签页渲染 ON/OFF 开关 (`id=__ps_adblk__`)，点击时通过 `tg()` 函数的 `onclick` 处理器设置全局变量 `window.__pakeAdblockRunning`。`adblock.js` 的 `shouldBlockUrl()` 和 `reportBlock()` 在每次执行前通过 `isAdblockEnabled()` 检查该标志——若为 `false`，所有拦截逻辑立即返回，请求正常通行。
- (7) 拦截统计展示。设置面板的拦截统计区域通过 `loadAdblockStats()` 调用 `adblock_get_stats` IPC 命令获取实时数据（拦截次数、规则数量）。托盘菜单的 tooltip 通过 `adblock::update_tray_block_count` 更新为“已拦截 N 个请求”。前端拦截计数器每拦截一次递增，同时调用 `adblock_report_blocked` IPC 通知后端同步更新。
- (8) 自定义规则生命周期。用户点击“编辑自定义规则”展开文本区，输入规则后点击 Save。`doSave()` 收集 `__ps_rules__` 文本区的值构造 `AppSettings.adblock.custom_rules`，调用 `save_settings` IPC。Rust 后端的 `save_settings` 命令在写入 JSON 文件后，检测广告规则是否变化——若有差异，依次调用 `remove_custom_rule` 清除旧规则、`add_custom_rule` 添加新规则，每个规则再次经 `parse_custom_rule` 校验后写入引擎的 `custom_lines` 列表，并重新合并生成网络规则和 CSS 规则。

4.1.3 关键代码

1. 规则解析与分类

Listing 2: EasyList 规则解析器

```

1 pub struct ParsedRules {
2     pub network: Vec<NetworkRule>,
3     pub cosmetic: Vec<CosmeticRule>,
4 }
5 pub fn parse_rules_text(text: &str) -> ParsedRules {
6     let mut network = Vec::new(); let mut cosmetic = Vec::new();
7     for line in text.lines() {
8         let trimmed = line.trim();

```

```
9         if trimmed.is_empty() || trimmed.starts_with('!') { continue;
10             }
11         if trimmed.starts_with("##") { cosmetic.push(CosmeticRule {
12             ... }); }
13         else if trimmed.starts_with("||") { network.push(NetworkRule
14             ::Domain(...)); }
15         else if trimmed.starts_with('/') { network.push(NetworkRule::
16             UrlRegex(...)); }
17     }
18     ParsedRules { network, cosmetic }
19 }
```

引擎启动时通过 `parse_rules_text` 将 EasyList 规则文本解析为结构化数据。域名规则 (`||domain^`) 存储为纯字符串用于 `hostname` 匹配；正则规则 (`/pattern/`) 编译为 `Regex` 对象并用 `Arc` 包装以支持跨线程共享；CSS 隐藏规则 (`##selector`) 存储为选择器字符串。

2. 配置注入与初始化顺序

Listing 3: AdblockState 初始化和窗口注入

```
1 // lib.rs setup closure — order matters!
2 let adblock_state = adblock::AdblockState::new(true, &adblock_rules);
3 app.manage(adblock_state); // MUST be before set_window
4
5 let window = set_window(app.app_handle(), &pake_config, &tauri_config
6     )?;
7 // Inside build_window():
8 if config.block_ads {
9     if let Some(state) = app.try_state::<adblock::AdblockState>() {
10         let export = state.engine.export_for_injection(None);
11         window_builder = window_builder
12             .initialization_script(&format!("window.pakeAdblock={}",
13                 json!(export)))
14             .initialization_script(include_str!("../inject/adblock.js
15                 "));
16     }
17 }
```

`app.manage()` 必须在 `set_window` 之前调用，否则 `build_window` 中的 `try_state` 返回 `None`，导致 `adblock.js` 不被注入。这是最常见的初始化顺序问题。

3. 前端拦截代理与运行时开关

Listing 4: adblock.js 核心逻辑

```
1 const originalFetch = window.fetch;
2 window.fetch = function (input, init) {
3   if (!isAdblockEnabled()) return originalFetch.apply(this,
4     arguments);
5   const url = typeof input === 'string' ? input : input?.url;
6   if (shouldBlockUrl(url)) { reportBlock(); return new Response('',
7     {status:204}); }
8   return originalFetch.apply(this, arguments);
9 };
10
11 function applyCosmeticRules() {
12   const css = config.cosmetic_selectors.map(s =>
13     s.replace(/~##/, '') + '{display:none!important}').join('\n')
14   ;
15   let style = document.getElementById('pake-adblock-style');
16   if (!style) { style = document.createElement('style'); style.id =
17     'pake-adblock-style';
18     (document.head || document.documentElement).appendChild(style
19       ); }
20   style.textContent = css;
21 }
22
23 const observer = new MutationObserver(() => applyCosmeticRules());
24 observer.observe(document.documentElement, { childList: true, subtree
25   : true });
```

4. 设置面板到引擎的规则同步

Listing 5: save_settings 中的规则同步

```
1 // In settings/commands.rs save_settings()
2 if let Some(state) = app.try_state::<crate::adblock::AdblockState>()
3 {
4   let current = state.engine.custom_rules();
5   if current != settings.adblock.custom_rules {
6     for rule in &current { state.engine.remove_custom_rule(rule);
7       }
8     for rule in &settings.adblock.custom_rules {
9       let _ = state.engine.add_custom_rule(rule);
10     }
11     eprintln!("[Pake] adblock rules synced: {} rules", settings.
```

```
10         adblock.custom_rules.len());
11     }
```

4.1.4 效果截图

- CLI 构建命令与 `--block-ads` 选项输出
- 左下角 Pake Plus 按钮与页面正常运行状态
- 设置面板 Adblock 标签——ON/OFF 开关、ON 标签、拦截统计数字、编辑自定义规则
- 自定义规则编辑区展开状态（文本区 + 格式提示）
- 托盘菜单 tooltip 显示“已拦截 N 个请求”

4.1.5 遇到的问题与解决方案

- (1) **AdblockState 初始化时机晚于窗口创建，导致 adblock.js 不注入**
→ 解决：将 `app.manage(adblock_state)` 移到 `set_window` 调用之前。初始化顺序为：创建 AdblockState → manage → set_window → build_window 中 try_state 获取状态 → 注入脚本。若顺序错误，`try_state` 返回 None 且无任何错误提示，通过 `eprintln!` 日志定位。
- (2) **设置面板自定义规则保存后不生效——JSON 存储与引擎内存脱节**
→ 解决：在 `save_settings` IPC 命令末尾增加规则同步逻辑。对比当前引擎中的 `custom_rules()` 与新设置中的规则列表，先清除所有旧规则再逐条添加新规则。每个规则添加前通过 `parse_custom_rule` 校验格式，非法规则不进入引擎。
- (3) **Tauri v2 ACL 拦截所有自定义 IPC 命令（adblock_get_stats 等无法调用）**
→ 解决：将 adblock 的命令名写入 `permissions/settings.toml` 的 `commands.allow` 列表，确保与 `capabilities/default.json` 中的权限标识符对应。Tauri v2 要求每个自定义命令显式授权，分散在多个权限文件容易遗漏。
- (4) **开关切换后拦截不停止——adblock.js 在加载时已固化状态**
→ 解决：在 `adblock.js` 中增加 `isAdblockEnabled()` 函数，在每次 `shouldBlockUrl` 和 `reportBlock` 执行前动态检查 `window.__pakeAdblockRunning` 全局变量。设置面板的开关 `onclick` 和 `fillAdblock` 加载时同步更新该变量。
- (5) **多个齿轮按钮并存（旧右下角 FAB + 新左下角按钮）**
→ 解决：定位 `custom.js` 中两处按钮创建代码——旧 FAB (`__ps_fab`) 和旧 `__pgb`——逐一移除，统一使用左下角白色卡片样式按钮。

- (6) 自定义规则文本区默认不可见——用户不知道如何编辑
- 解决：改为折叠面板设计——默认显示“ 编辑自定义规则” 按钮（虚线边框），点击后展开文本区并切换为“ 收起”（蓝色实线边框），提供清晰的展开/收起视觉反馈。

4.2 离线缓存代理模块（成员 B）

4.2.1 设计思路

- 目标：在 HTTP 层透明代理请求，将响应缓存到磁盘，支持离线访问
- Rust 技术：异步 I/O（tokio）、资源拦截、LRU 缓存淘汰、HTTP 头解析
- 核心挑战：大量并发请求的缓存写入吞吐量；缓存过期策略的准确性；断网判断

4.2.2 核心流程

- (1) 注册 Tauri HTTP 拦截器，拦截所有 GET 请求
- (2) 请求到达时查本地缓存索引，命中且未过期直接返回缓存数据
- (3) 未命中或已过期则放行请求，获取响应后异步写入磁盘
- (4) 维护缓存文件索引（URL 哈希 → 文件路径 → 大小 → 最后访问时间）
- (5) 缓存总量超过上限（默认 200MB）时触发 LRU 淘汰
- (6) 断网时展示离线提示页，列出可访问的已缓存页面

4.2.3 关键代码

Listing 6: 离线缓存示例代码

1

% TODO：成员 B 插入核心代码片段（HTTP 拦截器、LRU 淘汰、缓存索引维护等）

2

% 建议 1-2 段关键代码，每段 5-20 行

4.2.4 效果截图

4.2.5 遇到的问题与解决方案

4.3 剪贴板管理模块（成员 C）

4.3.1 设计思路

剪贴板管理模块的目标是在不打断用户原有复制、粘贴习惯的前提下，为 Pake 应用补充可检索、可复用的剪贴板历史。模块采用“系统监听层—隐私过滤层—持久化层—交互层”的分层结构：监听层通过 `clipboard_rs` 对接 Windows、macOS 和 X11 的原生剪贴板事件，在纯 Wayland 环境下使用 300 ms 低频轮询作为兼容方案；监听任务运行在独立线程中，避免阻塞 Tauri 主线程。每次检测到文本变化后先计算 SHA-256 摘要，并结合 `last_hash` 与 `expected_hash` 消除连续重复内容以及“从历史记录复制”造成的回写。

通过初步去重的文本还需经过隐私过滤：空文本、过短或过长文本、疑似密码、通过 Luhn 校验的银行卡号，以及用户指定应用产生的内容均可跳过。有效记录存入 SQLite 数据库，数据库启用 WAL 模式并为内容哈希和创建时间建立索引；相同内容再次出现时通过 UPSERT 更新时间，从而兼顾去重和最近使用排序。前端使用独立的 Tauri Webview 面板，通过 IPC 完成列表加载、关键词搜索、复制、删除、清空和设置更新。该设计使监听、存储和界面相互解耦，同时兼顾跨平台能力、性能与隐私安全。

4.3.2 核心流程

- (1) **初始化配置与运行状态。**应用启动时，`init_clipboard_state` 首先通过 `get_data_dir` 获取应用数据目录，并定位 `clipboard-settings.json`。随后调用 `ClipboardSettings::load` 读取持久化设置；若文件不存在或 JSON 损坏，则回退到构建参数 `clipboard` 与 `clipboard_max` 所确定的默认值。`normalized` 会再次执行构建开关校验，并将最大记录数限制在 500–5000 条之间，防止运行时配置绕过打包阶段的能力限制。设置保存采用“先写临时文件，再重命名替换”的方式，以降低程序异常退出造成配置文件只写入一半的风险。
- (2) **创建 SQLite 历史数据库。**仅当剪贴板功能启用时，后端才调用 `ClipboardStore::open`，在 `clipboard/history.db` 中创建数据库。数据库表保存编号、文本内容、SHA-256 摘要、创建时间和来源应用五类信息，并启用 WAL 日志模式。程序为 `content_hash` 建立唯一索引，为 `created_at` 建立普通索引；启动时还会检测旧数据库是否缺少 `source_app` 字段，并通过 `ALTER TABLE` 自动迁移。数据库打开后立即执行一次清理，避免过期数据在应用重新启动后继续保留。
- (3) **选择与启动平台监听方式。**`start_monitor` 先判断当前 Linux 会话是否为纯 Wayland：当 `WAYLAND_DISPLAY` 存在且 `DISPLAY` 为空时，启动 `wl_clipboard_rs`

轮询线程，每 300 ms 读取一次文本；其他环境则通过 `clipboard_rs` 创建 `ClipboardWatcherContext`，注册 `HistoryHandler` 接收原生剪贴板变化事件。监听启动前会读取一次当前剪贴板并保存摘要，因而不会把应用启动前已经存在的内容误记为新记录。监听线程同时保存关闭通道或原子停止标志，在模块停用和应用退出时可以停止循环并 `join` 回收线程。

- (4) **读取变化并执行双重去重。**系统通知到达后，`HistoryHandler::on_clipboard_change` 调用 `get_text` 获取纯文本，并通过 `ClipboardStore::hash_content` 计算 SHA-256。第一层去重检查共享的 `expected_hash`：若该摘要来自历史面板主动写回，则消费标记并直接返回；第二层检查监听器自己的 `last_hash`，过滤操作系统可能连续发送的相同通知。只有两次检查均未命中的文本才会进入 `process_text`，从源头减少过滤和数据库写入开销。
- (5) **识别剪贴板来源应用。**去重完成后，后端调用 `current_application` 获取来源名称。Windows 优先读取 `GetClipboardOwner`，若应用已释放所有权则回退到前台窗口，再根据进程编号查询可执行文件名；macOS 通过 `NSWorkspace` 获取最前端应用；X11 根据 `_NET_ACTIVE_WINDOW` 和 `_NET_WM_PID` 查找进程名称。纯 Wayland 出于协议权限限制无法获知其他客户端的来源，因此允许该字段为空。
- (6) **执行隐私与有效性过滤。**`process_text` 先从 `RwLock` 读取最新设置：模块已关闭时立即结束。随后构造 `FilterConfig` 并调用 `should_record`：空文本和少于 2 个字符的短文本默认跳过，超过 10000 个字符的内容拒绝记录；来源应用名称会去除路径及 `.exe/ .app` 后缀，再与忽略列表进行不区分大小写的比较；13–19 位数字候选值只有通过 Luhn 校验才按银行卡号过滤；6–30 位、无空白且同时包含字母和数字的内容按疑似密码处理。这些隐私开关可在本地历史面板中修改，过滤失败的内容不会进入数据库。
- (7) **持久化记录并保持最近使用顺序。**合格文本交给 `insert_with_source`。该函数再次计算摘要并生成 UTC RFC 3339 时间，然后使用参数化 SQL 写入数据库。若唯一索引发现相同 `content_hash` 已存在，`ON CONFLICT DO UPDATE` 不新增行，而是刷新文本、时间和可用的来源应用；因此重复复制的内容只会移动到历史列表顶部，不会无限产生副本。查询列表时按 `created_at DESC`，`id DESC` 排序，默认返回最近 100 条，IPC 层把允许的返回数量限制在 1–500 条。
- (8) **定期清理过期与超量数据。**每次插入结束后都会调用 `cleanup`，先删除早于保留期限的记录；默认保留 30 天。若剩余数量超过 `max_items`，则按创建时间升序选出最早记录，并以 500 条为批次淘汰，避免为每一条超限数据启动一次删除事务。除此之外，`start_cleanup` 还创建名为 `pake-clipboard-cleanup` 的后台线程，通过 `recv_timeout` 每小时执行一次清理，即使长时间没有新的复制操作，过期记录也能被移除；停止信号到达后线程立即退出。

- (9) **通过全局快捷键或设置面板打开历史面板。**功能启用后, `set_global_shortcut` 在 Windows/Linux 注册 `Ctrl+Shift+V`, 在 macOS 注册 `Cmd+Shift+V`。若它与应用唤醒快捷键相同, 则剪贴板历史优先; 回调还通过 `Instant` 设置 300 ms 防抖。此外, 设置面板的剪贴板标签页提供蓝色“打开剪贴板历史”按钮, 点击后通过 Tauri 事件系统 (`emit` → `listen`) 通知 Rust 后端打开面板。触发后 `toggle_panel` 创建或复用 `clipboard-panel` Webview: 初始大小为 420×520 , 最小为 320×360 , 窗口带标题栏、居中显示、可缩放且显示在任务栏中。关闭按钮隐藏窗口而非销毁, 面板已存在时只刷新内容并获取焦点。
- (10) **加载列表并完成多关键词搜索。**面板显示后调用 `clipboard_list` 加载最近记录, 同时读取 `clipboard_stats` 展示“当前条数/容量上限”。用户输入搜索内容时, 前端先等待 120 ms, 以减少连续输入造成的 IPC 请求; 随后调用 `clipboard_search`。后端按空白拆分关键词, 对%、_ 和反斜杠进行转义, 再为每个词生成 `LOWER(content) LIKE` 条件, 并用 AND 连接。因此输入多个关键词时, 记录必须同时包含所有词才会命中; 所有值均通过 SQL 参数绑定传入, 避免查询语义被特殊字符改变。
- (11) **复用、删除和清空历史。**用户点击记录或“复制”按钮后, 前端将记录编号传给 `clipboard_copy_item`; 后端先按 ID 读取完整文本, 再由 `write_text_to_clipboard` 写回系统剪贴板。写入前保存 `expected_hash`, 写入失败时立即撤销, 成功后若 1 秒内没有对应通知也会自动清除, 防止标记长期残留。复制成功后面板保持打开, 并在鼠标附近显示 3 秒提示。单条删除调用 `clipboard_delete_item`; 清空全部记录必须先通过面板内确认框, 再调用 `clipboard_clear_all`, 避免误操作。
- (12) **运行时更新设置。**用户保存隐私设置后, `clipboard_update_settings` 会先归一化并持久化新值, 再依次停止旧监听线程和清理线程, 更新共享 `RwLock`。若功能仍启用, 则重新设置数据库容量并启动新的监听与清理任务; 若关闭, 则释放数据库状态。最后同步更新系统托盘中的启用状态, 保证面板设置、后台任务和托盘显示一致。

4.3.3 关键代码

1. 监听回调与双重去重

Listing 7: 剪贴板变化监听

```

1 impl ClipboardHandler for HistoryHandler {
2     fn on_clipboard_change(&mut self) {
3         if let Ok(text) = self.clipboard.get_text() {
4             let hash = ClipboardStore::hash_content(&text);

```

```

5         if consume_expected_hash(&self.expected_hash, &hash) {
6             self.last_hash = Some(hash);
7             return;
8         }
9         if self.last_hash.as_deref() == Some(&hash) { return; }
10        self.last_hash = Some(hash);
11        let source = current_application();
12        process_text(&text, source.as_deref(),
13                    &self.store, &self.settings, &self.expected_hash);
14    }
15 }
16 }

```

监听器取得文本后首先消费 `expected_hash`，避免用户从历史面板复制内容时再次入库；随后用 `last_hash` 过滤系统可能重复发送的变化通知。只有真正的新内容才进入隐私过滤和存储流程。

2. SQLite 哈希去重与时间刷新

Listing 8: 剪贴板历史持久化

```

1 let hash = Self::hash_content(text);
2 let now = Utc::now().to_rfc3339();
3 let conn = self.lock()?;
4 conn.execute(
5     "INSERT INTO clipboard_history
6         (content, content_hash, created_at, source_app)
7     VALUES (?1, ?2, ?3, ?4)
8     ON CONFLICT(content_hash) DO UPDATE SET
9         content = excluded.content,
10        created_at = excluded.created_at,
11        source_app = COALESCE(excluded.source_app,
12                               clipboard_history.source_app)",
13     params![text, hash, now, source_app],
14 )?;

```

数据库在 `content_hash` 上建立唯一索引。再次复制相同文本时不创建重复行，而是更新时间和来源应用，使该记录回到列表顶部。SQLite WAL 模式还可以降低监听线程写入与面板查询之间的相互阻塞。

3. 多关键词全文搜索

Listing 9: 剪贴板历史搜索条件构造

```

1 let keywords: Vec<String> = query.split_whitespace()

```

```
2     .map(str::trim)
3     .filter(|part| !part.is_empty())
4     .map(|part| escape_like_pattern(&part.to_lowercase()))
5     .collect();
6
7 for index in 0..keywords.len() {
8     if index > 0 { sql.push_str(" AND "); }
9     sql.push_str(&format!(
10         "LOWER(content) LIKE ?{} ESCAPE '\\'", index + 1));
11 }
```

搜索词按空白拆分，多个关键词必须同时命中；%、_ 和反斜杠会先被转义，查询值再通过参数绑定传入，从而避免用户输入改变 SQL 语义。

4.3.4 效果截图

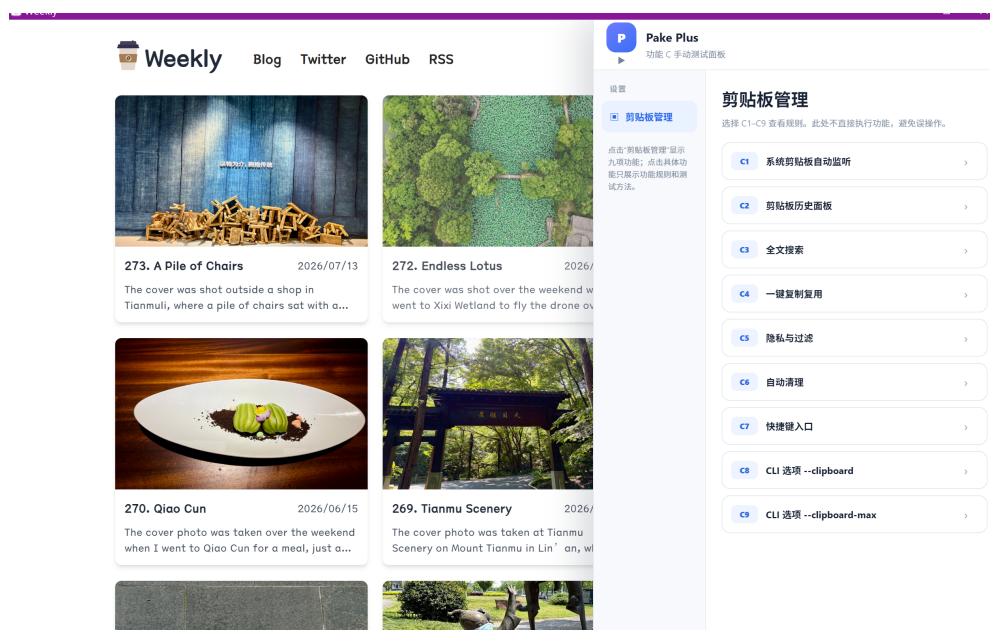
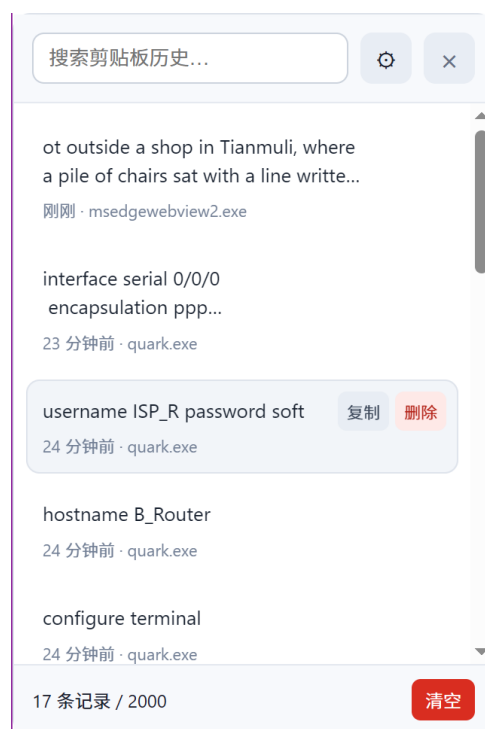


图 1: 剪贴板管理功能总览



(a) 功能测试说明



(b) 剪贴板历史列表

图 2: 功能说明与剪贴板历史面板

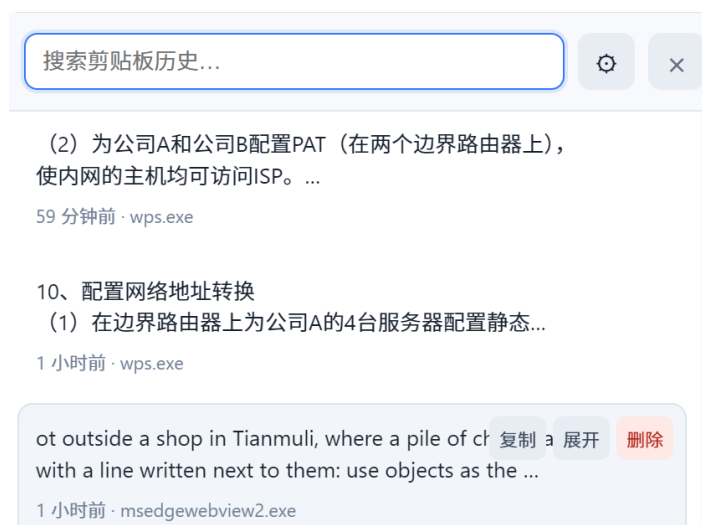


图 3: 外部程序复制的历史记录



(a) 隐私过滤设置

(b) 历史记录搜索

图 4: 隐私过滤与历史记录搜索



图 5: 长文本展开效果

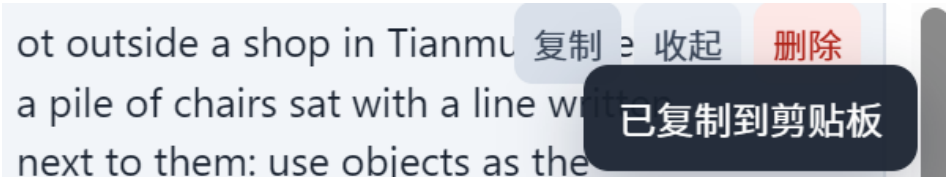


图 6: 一键复制成功提示



图 7: 清空全部剪贴板历史确认框

273. A Pile of Chairs

图 8: 全局搜索快捷键入口

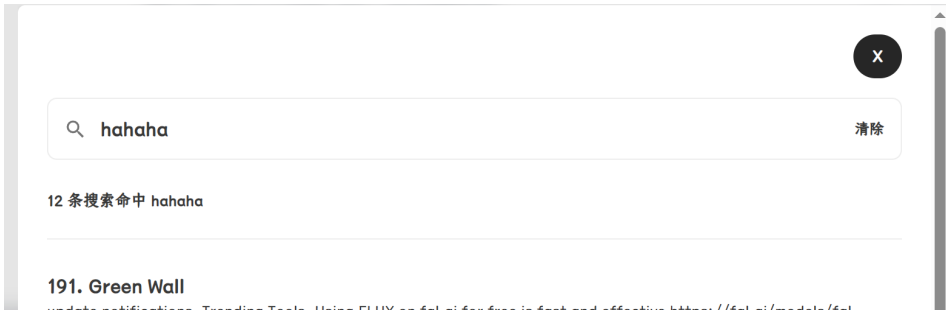


图 9: 关键词搜索结果展示

4.3.5 遇到的问题与解决方案

- (1) 不同桌面系统的剪贴板机制不统一
→解决: Windows、macOS 和 X11 优先使用 clipboard_rs 提供的原生事件监听; 纯 Wayland 无法沿用 X11 事件接口, 因此使用 wl_clipboard_rs 每 300 ms 读取一次, 并通过原子停止标志安全结束线程。
- (2) 从历史面板复制时会触发监听器, 造成记录再次写入
→解决: 写入系统剪贴板前保存目标文本的 SHA-256 到 expected_hash; 下一次监听回调命中该摘要后立即消费并跳过, 1 秒内未触发事件则自动清除标记。
- (3) 重复通知和重复内容导致历史列表膨胀

→解决：内存中的 `last_hash` 过滤连续通知，数据库中的唯一哈希索引与 UPSERT 负责持久化去重；重复复制只刷新时间，不新增记录。

(4) 剪贴板可能包含密码、银行卡号等敏感信息

→解决：存储前增加可配置过滤链。疑似密码按长度、空白、字母和数字组合判断；银行卡候选值必须满足 13–19 位并通过 Luhn 校验；同时支持按来源应用忽略记录。

(5) 历史记录持续增长会影响磁盘占用和查询性能

→解决：默认最多保存 2000 条并保留 30 天。写入后立即执行容量检查，后台每小时执行一次清理；超限时按最早时间以 500 条为批次删除，减少频繁的小事务。

(6) 全局快捷键可能重复触发或与应用唤醒快捷键冲突

→解决：注册时检测冲突并让剪贴板快捷键优先；事件处理增加 300 ms 防抖。面板采用居中、带标题栏的独立 Webview 窗口，标题栏 × 关闭按钮隐藏窗口而非销毁，重复按快捷键或点击设置面板按钮可重新打开。

4.4 设置面板与数据管理模块（成员 D）

4.4.1 设计思路

设置面板是整个 Pake Plus 的”控制中心”，负责统一管理所有模块的配置。核心设计原则是各模块独立运行，配置集中管理——即使面板未打开，各模块以默认配置正常运行；用户通过面板修改后即时写入本地 JSON，各模块下次读取时自动生效。

面板采用纯 JavaScript DOM 操作构建，无任何框架依赖。所有 UI 元素通过封装的 `el()` / `card()` / `row()` / `tg()` 工厂函数动态生成，支持中英文双语切换。Rust 后端负责 JSON 读写、ZIP 打包解压、系统信息采集和配置校验，通过 Tauri IPC 与前端双向通信。

前端代码(`custom.js`)以 IIFE 形式注入主窗口，在页面加载完成后通过 `setTimeout` 延迟创建左下角的齿轮入口按钮。点击按钮时动态创建包含 6 个标签页的右侧抽屉面板，各标签页通过 `buildXxx(body)` 函数按需渲染。配置数据存储在 `pake-settings.json` 中，保存前自动轮转保留最近 5 个历史版本。

4.4.2 核心流程

1. 面板入口与生命周期。应用启动 800ms 后，`custom.js` 在页面左下角创建一个白色圆角卡片按钮（`position:fixed;bottom:24px;left:24px`），包含蓝色渐变齿轮图标和”Pake Plus”文字，hover 时边框高亮。用户点击按钮、右键托盘选择 Settings、或按 `Ctrl+Shift+,` 均可打开面板。面板为 `position:fixed;top:0;right:0;width:400px` 的固定定位抽屉，初始状态通过 `transform:translateX(100%)` 隐藏在屏幕右侧，打开

时 CSS transition 滑入。面板包含左侧 120px 导航栏和右侧内容区，6 个标签页通过 `buildXxx(body)` 函数按需渲染。

2. 配置加载与表单填充。面板打开后调用 `get_settings` IPC 命令加载 `AppSettings` 结构体。该结构聚合四个子模块的配置 (`adblock` / `cache` / `clipboard` / `general`)，每个子模块独立实现 `ModuleSettings` trait 提供 `validate()` / `summary()` / `stats()` 三个方法。加载完成后调用 `fillAll()` 遍历各标签页的 DOM 元素，通过 `tglSet()` 设置开关状态、`value` 填充选择框和文本区。

3. 表单校验与保存。用户修改任何表单元素后 `dirty` 标志置为 `true`。点击 `Save` 时，`doSave()` 收集各标签页表单值构造完整 `AppSettings` 对象，调用 `validate_settings` 校验参数合法性（如缓存范围 50-1000MB、剪贴板记录数在允许列表内等）。校验通过后调用 `save_settings` 写入磁盘，同时触发广告拦截规则同步——后端比较新旧自定义规则，调用 `add_custom_rule` / `remove_custom_rule` 更新引擎。

4. 数据导出。用户点击”导出”或输入路径后点”浏览”使用原生保存对话框。JS 调用 `export_data` IPC (可选 `savePath` 参数)，Rust 递归扫描 `AppData` 目录含子目录，生成 `manifest.json` 记录文件清单和元数据，用 `zip crate` 打包为 `pake-data-YYYYMMDD.zip`。

5. 数据导入。用户指定 ZIP 路径或使用原生文件选择器（通过 `rfd crate` 的 `pick_zip_file` 命令），JS 调用 `preview_import` 展示内容摘要，用户确认后调用 `import_data`。Rust 解压到临时目录，按 `manifest` 路径映射逐文件恢复，现有文件先备份为 `.bak`。

6. 系统诊断。`About` 标签调用 `get_diagnostics` IPC。Rust 使用 `built crate`（编译期注入）获取版本、Git 哈希、编译时间等；使用 `sysinfo crate` 采集 OS、CPU、内存、磁盘信息。点击”复制诊断报告”通过 `arboard crate` 写入系统剪贴板。

7. 版本备份与恢复。`write_settings` 在保存前执行备份轮转：当前文件 → `v1`，旧 `v1`→`v2`，...，`v4`→`v5`，始终保留 5 个版本。启动时 `run_health_check` 检测配置是否可解析，若损坏自动从最近备份恢复。

4.4.3 关键代码

1. Rust 枚举类型（类型安全）

Listing 10: Theme 和 Language 枚举

```
1 #[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
2 #[serde(rename_all = "lowercase")]
3 pub enum Theme {
4     Light,
5     Dark,
6     System,
7 }
8
```

```
9 #[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
10 #[serde(rename_all = "lowercase")]
11 pub enum Language {
12     #[serde(alias = "zh-CN")]
13     Zh,
14     En,
15 }
```

2. ModuleSettings Trait (多态)

Listing 11: ModuleSettings trait

```
1 pub trait ModuleSettings {
2     fn validate(&self) -> Vec<String>;           // 校验输入
3     fn summary(&self) -> String;                 // 一行摘要
4     fn stats(&self) -> serde_json::Value;       // 统计 JSON
5 }
6
7 // 每个模块独立实现
8 impl ModuleSettings for AdblockSettings { ... }
9 impl ModuleSettings for CacheSettings { ... }
10 impl ModuleSettings for ClipboardSettings { ... }
11 impl ModuleSettings for GeneralSettings { ... }
```

3. 配置校验

Listing 12: 设置校验逻辑

```
1 #[command]
2 pub fn validate_settings(settings: AppSettings) -> Result<(), Vec<
3     String>> {
4     let mut errors: Vec<String> = Vec::new();
5     errors.extend(settings.adblock.validate()); // 规则格式
6     errors.extend(settings.cache.validate());   // 缓存范围
7     errors.extend(settings.clipboard.validate()); // 记录数/天数合法性
8     errors.extend(settings.general.validate()); // 枚举类型编译期保证
9     if errors.is_empty() { Ok(()) } else { Err(errors) }
10 }
```

4. 配置校验、备份轮转与规则同步

Listing 13: 配置校验、备份轮转与广告规则同步

```

1  #[command]
2  pub fn validate_settings(settings: AppSettings) -> Result<(), Vec<
    String>> {
3      let mut errors = Vec::new();
4      errors.extend(settings.adblock.validate());
5      errors.extend(settings.cache.validate());
6      errors.extend(settings.clipboard.validate());
7      errors.extend(settings.general.validate());
8      if errors.is_empty() { Ok(()) } else { Err(errors) }
9  }
10
11 const MAX_BACKUPS: u32 = 5;
12 fn write_settings(app: &AppHandle, s: &AppSettings) -> Result<(),
    String> {
13     if path.exists() { /* v1→v2, ..., v4→v5 rotation */ }
14     fs::write(&path, &serde_json::to_string_pretty(s).unwrap())?;
15     // Sync adblock custom rules to running engine
16     if let Some(st) = app.try_state::<crate::adblock::AdblockState>()
        {
17         let cur = st.engine.custom_rules();
18         if cur != s.adblock.custom_rules {
19             for r in &cur { st.engine.remove_custom_rule(r); }
20             for r in &s.adblock.custom_rules { st.engine.
                add_custom_rule(r); }
21         }
22     }
23     Ok(())
24 }

```

5. 原生文件对话框

Listing 14: rfd 文件选择 IPC

```

1  #[command] pub fn pick_save_path() -> Result<String, String> {
2      rfd::FileDialog::new().set_title("选择导出位置")
3          .save_file().map(|p| p.to_string_lossy().to_string())
4          .ok_or_else(|| "cancelled".into())
5  }
6  #[command] pub fn pick_zip_file() -> Result<String, String> {
7      rfd::FileDialog::new().set_title("选择数据文件")

```

```
8         .add_filter("ZIP", &["zip"]).pick_file()
9         .map(|p| p.to_string_lossy().to_string())
10        .ok_or_else(|| "cancelled".into())
11    }
```

4.4.4 效果截图

- 左下角 Pake Plus 入口按钮（白色卡片 + 蓝色边框 + 齿轮图标）
- 一般标签页——暗色主题开关、语言下拉、恢复默认按钮
- 广告拦截标签页——ON/OFF 开关 + 标签、 编辑自定义规则折叠面板
- 剪贴板标签页——开关、记录数选择、保留天数、忽略短文本
- 数据管理标签页——导出/导入按钮、路径输入框、浏览按钮
- 关于标签页——诊断信息（应用信息 + 系统信息）、复制报告按钮
- 托盘菜单——Settings、Clipboard History、Copy Diagnostics

4.4.5 遇到的问题与解决方案

- (1) **Tauri initializtion_script** 中 **document.body** 为 **null**
→ 解决：将 DOM 操作包裹在 **DOMContentLoaded** 事件中，并增加即时检查兜底
- (2) 切换标签页后保存导致其他标签页设置丢失
→ 解决：doSave 函数先检查对应元素是否存在于 DOM，不存在则沿用当前 S 状态的值
- (3) **built crate** 未启用 **git2 feature** 导致 **GIT_COMMIT_HASH** 不可用
→ 解决：在 Cargo.toml 中添加 `built = { version = "0.7", features = ["git2"] }`
- (4) 旧设置文件导致新默认值不生效
→ 解决：删除旧文件（AppData/Roaming 而非 Local），启动健康检查自动从备份恢复损坏文件
- (5) **Theme/Language** 使用 **String** 类型无法编译期保证合法性
→ 解决：重构为 Rust 枚举类型，`#[serde(rename_all = "lowercase")]` 保证序列化兼容

- (6) **Tauri v2 ACL 拦截自定义 IPC 命令**
→ 解决: 将所有 app 级命令统一放在 `permissions/settings.toml` 的 `commands.allow` 列表中, 而非分散在多个权限文件, 确保所有命令被正确授权
- (7) **About 标签诊断信息不显示**
→ 解决: 在 `buildAbout` 末尾添加 `loadDiagnostics()` 调用, 截断 Git 哈希至 8 位, 添加 `word-break:break-all` 防止文字溢出
- (8) **右下角旧齿轮按钮与新左下角按钮并存**
→ 解决: 定位到原始代码中的 FAB (Floating Action Button) `__ps_fab` 和旧 `__pgb` 按钮代码, 逐一从 `custom.js` 中移除

5 实验结果

5.1 测试数据

测试类型	测试数量	通过	结果
Rust 后端测试	67	67	全部通过
JavaScript 测试	297	297	全部通过
cargo check	—	—	0 warnings
cargo build	—	—	成功

表 2: 测试结果汇总

5.2 关键测试用例

针对成员 C 的 CLI 参数、配置合并、面板交互与权限控制进行了定向测试, 共执行 5 个测试文件、56 个测试用例, 结果全部通过。

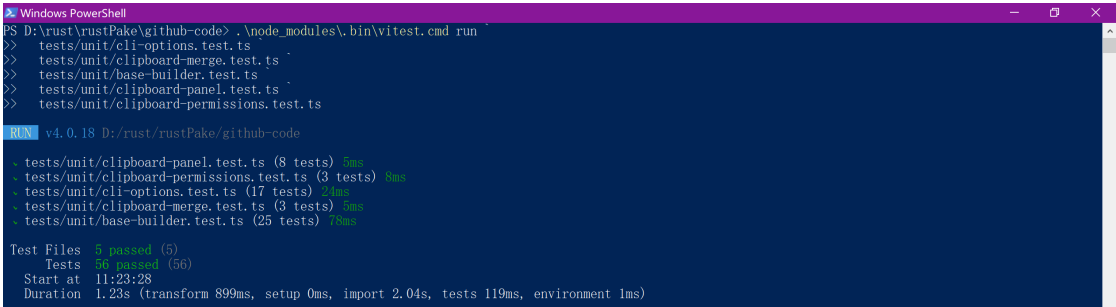


图 10: PowerShell A: 成员 C 定向测试结果 (56 项全部通过)

测试用例	结果
默认设置合理性检查	通过
JSON 序列化/反序列化往返	通过
部分 JSON 解析（缺失字段回退默认值）	通过
空 JSON 解析	通过
广告拦截规则序列化	通过
Theme 枚举序列化为小写	通过
Theme 枚举反序列化（合法值 + 非法值拒绝）	通过
广告规则格式校验（合法 + 非法）	通过
缓存大小范围校验（越界 + 合法）	通过
剪贴板参数校验（非法值 + 合法值）	通过
设置整体校验命令	通过
诊断信息字段完整性	通过

表 3: 关键测试用例

CLI 边界测试分别传入 499 和 5001,程序均在打包前拒绝参数,验证 `--clipboard-max` 仅接受 500-5000 范围内的整数。

```

PS D:\rust\rustPake\github-code> Set-Location D:\rust\rustPake\github-code
PS D:\rust\rustPake\github-code>
PS D:\rust\rustPake\github-code> $env:CARGO_HOME="D:\rust\rustPake\local-only\cargo-home"
PS D:\rust\rustPake\github-code> $env:RUSTUP_HOME="D:\rust\rustPake\local-only\rustup-home"
PS D:\rust\rustPake\github-code> $env:COREPACK_HOME="D:\rust\rustPake\local-only\corepack"
PS D:\rust\rustPake\github-code> $rustToolchain="D:\rust\rustPake\local-only\rustup-home\toolchains\1.93.0-x86_64-pc-windows-msvc\bin"
PS D:\rust\rustPake\github-code> $pnpmShim="D:\rust\rustPake\local-only\bin"
PS D:\rust\rustPake\github-code> $env:Path="$rustToolchain;$env:CARGO_HOME\bin;$pnpmShim;D:\Node.js_MBTI\qoder;$env:Path"
PS D:\rust\rustPake\github-code> rustc --version
rustc 1.93.0 (25459607 2026-01-19)
PS D:\rust\rustPake\github-code> cargo --version
cargo 1.93.0 (083ac5135 2025-12-15)
PS D:\rust\rustPake\github-code> pnpm --version
10.26.2
PS D:\rust\rustPake\github-code> node --version
v24.15.0
PS D:\rust\rustPake\github-code> node .\dist\dev.js "https://example.com"
>> --name ClipboardInvalidLow
>> --clipboard
>> --clipboard-max 499
file:///D:/rust/rustPake/github-code/dist/dev.js:2861
    throw new Error(`--clipboard-max must be an integer between 500 and 5000`);
          ^
Error: --clipboard-max must be an integer between 500 and 5000
    at Option.parseArg (file:///D:/rust/rustPake/github-code/dist/dev.js:2861:19)
    at Command.callParseArg (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:601:21)
    at handleOptionValue (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:702:20)
    at Command.<anonymous> (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:722:7)
    at Command.emit (node:events:509:28)
    at Command.parseOptions (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:1799:18)
    at Command.parseCommand (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:1551:25)
    at Command.parse (D:\rust\rustPake\github-code\node_modules\pnpm\commander@14.0.3\node_modules\commander\lib\command.js:1093:10)
    at file:///D:/rust/rustPake/github-code/dist/dev.js:2901:9
    at ModuleJob.run (node:internal/modules/esm/module_job:437:25)
Node.js v24.15.0
PS D:\rust\rustPake\github-code> node .\dist\dev.js "https://example.com"
>> --name ClipboardInvalidHigh
>> --clipboard
>> --clipboard-max 5001

```

图 11: PowerShell A: 剪贴板容量参数边界校验

PowerShell B 进一步验证运行时默认值与容量压力场景。未指定容量上限的实例在空历史状态下显示“0 条记录 / 2000”，与默认值 2000 一致；随后通过带重试的 PowerShell 函数连续写入 500 条不同的普通中文文本，用于观察监听完整性及容量上限行为。

```

0 条记录 / 2000 清空
PS D:\rust\rustPake\github-code> function Set-ClipboardWithRetry {
>> param(
>> [Parameter(Mandatory)]
>> [string]$text
>> )
>> for ($attempt = 1; $attempt -le 50; $attempt++) {
>>     try {
>>         Set-Clipboard -Value $text -ErrorAction Stop
>>         return
>>     } catch {
>>         Start-Sleep -Milliseconds 100
>>     }
>> }
>> throw "剪贴板连续重试50次仍写入失败: $text"
PS D:\rust\rustPake\github-code>
PS D:\rust\rustPake\github-code> for ($i = 1; $i -le 500; $i++) {
>>     $text = "剪贴板容量测试第(0:$i)条, 这是一段普通中文测试文本" -f $i
>>     Set-ClipboardWithRetry -text $text
>>     if ($i % 50 -eq 0) {
>>         Write-Host "已成功写入 $i / 500 条"
>>     }
>>     Start-Sleep -Milliseconds 300
>> }
已成功写入 50 / 500 条
已成功写入 100 / 500 条
已成功写入 150 / 500 条
已成功写入 200 / 500 条
已成功写入 250 / 500 条
已成功写入 300 / 500 条
已成功写入 350 / 500 条
已成功写入 400 / 500 条
已成功写入 450 / 500 条
已成功写入 500 / 500 条
PS D:\rust\rustPake\github-code>
PS D:\rust\rustPake\github-code> Start-Sleep -Seconds 1
PS D:\rust\rustPake\github-code> Write-Host "500条测试内容已全部写入"
500条测试内容已全部写入
PS D:\rust\rustPake\github-code>
PS D:\rust\rustPake\github-code> Set-ClipboardWithRetry -text "剪贴板容量测试第0501条, 这是一段普通中文测试文本"
PS D:\rust\rustPake\github-code> Start-Sleep -Seconds 1

```

图 12: PowerShell B: 默认容量显示与 500 条剪贴板批量写入测试

5.3 命令行示例

Listing 15: CLI 使用示例

```
1 # 基本打包
2 pake https://github.com --name GitHub
3
4 # 启用所有 Pake Plus 功能
5 pake https://example.com --name MyApp \
6   --block-ads --adblock-rules ./my-rules.txt \
7   --cache --cache-size 500 \
8   --clipboard --clipboard-max 5000
```

6 开源项目参考说明

本项目基于开源项目 **Pake** (<https://github.com/tw93/Pake>) 进行二次开发。

6.1 引用部分

- Tauri v2 框架及窗口管理、系统托盘、全局快捷键能力
- CLI 打包工具链 (TypeScript + Rollup)
- WebView 注入机制 (initialization_script)
- 平台适配配置 (tauri.conf.json、pake.json)

6.2 新增部分 (对比原项目)

7 总结与展望

7.1 项目总结

Pake Plus 成功在 Pake 的基础上实现了四个独立功能模块：

- 广告拦截引擎基于全球最主流的 EasyList 规则集，网络层 + DOM 层双重过滤
- 离线缓存在 HTTP 层透明代理，实现断网可浏览
- 系统剪贴板管理通过 Rust FFI 调用操作系统 API，跨平台兼容
- 可视化设置面板统一管理所有配置，支持数据一键迁移和系统诊断

模块	新增内容	核心技术	代码量
广告拦截	EasyList 规则引擎、网络拦截、DOM 隐藏、自定义规则	请求拦截器	—
离线缓存	HTTP 代理缓存、LRU 淘汰、离线模式、缓存统计	tokio 异步 I/O	—
剪贴板	系统级监听 (FFI)、历史搜索、一键复用、自动清理	Win32 FFI	—
设置面板	可视化配置、数据导入导出、诊断采集、版本回滚、健康检查	ModuleSettings trait	~660 行

表 4: 与 Pake 原项目的差异

项目技术选型合理：Rust 负责系统级操作（文件 I/O、ZIP 打包、系统信息采集、FFI），JavaScript 负责用户界面，Tauri IPC 作为两者之间的桥梁。四个模块通过共享 JSON 配置文件实现了解耦协作。

7.2 不足与改进方向

- (1) **错误处理**：当前大量使用 `Result<(), String>`，可通过 `thiserror` 库定义结构化错误类型
- (2) **异步 I/O**：IPC 命令中文件操作使用同步 I/O，可能阻塞事件循环，可迁移至 `tokio::fs`
- (3) **托盘菜单动态更新**：当前无法在运行时更新菜单项文字，需重建托盘
- (4) **打包时预设配置**：CLI 尚不支持将设置写入打包产物，需增加 `---settings` 选项
- (5) **测试覆盖**：JS 端缺少设置面板相关的自动化测试